

Project ID:

J26-SE-334

1. Topic (12 words max)

Human Behavior-Aware Robustness Testing Framework for Mobile Applications

2. Research group the project belongs to

SST - Software Systems & Technologies

3. Specialization of the project belongs to

Software Engineering (SE)

4. If a continuation of a previous project:

Project ID	
Year	

5. Brief description of the research problem including references (300 words max) – references not included in word count.

Mobile applications have become essential infrastructure for individuals, enterprises, and public institutions, supporting services such as banking, healthcare, transportation, and government operations. The primary user personas include everyday mobile users, software developers, quality assurance engineers, enterprises, and public organizations that depend on reliable mobile applications. However, despite extensive testing, many applications still fail under real-world operating conditions because existing testing frameworks assume ideal execution environments.

Current mobile testing approaches inadequately represent real-world environmental conditions. Studies have shown that runtime factors such as network behavior, energy consumption, and device fragmentation significantly influence application reliability [1], [2], [4], [5], while resource-monitoring techniques mainly observe these conditions rather than actively reproducing them [3]. Recent AI-driven chaos testing further highlights the importance of fault injection, but lacks coordinated environmental perturbation across multiple runtime conditions [6].

Similarly, existing testing tools assume perfect human interaction, whereas real users perform inaccurate touches, irregular gestures, repeated taps, and stress-driven actions. Human-computer interaction research demonstrates that touch accuracy depends on movement characteristics and finger behavior [7], [8], [9], while emotional states

significantly influence interaction patterns [10], [11]. Although recent AI-based mobile testing improves exploration strategies [12], automated testing still struggles to reproduce realistic human behavior and uncover many real-world GUI failures [13].

Another limitation is the inability to detect non-crash functional failures, such as overlapping layouts, hidden buttons, frozen loading screens, and broken interfaces. Recent vision-based and multimodal testing approaches improve UI verification [14]–[19], but most rely on cloud-based models, introducing latency, privacy concerns, and operational costs.

Furthermore, production failures rarely translate automatically into reusable regression tests. Existing crash reproduction frameworks depend on manually written bug reports [20]–[22], while semantic embeddings and self-healing techniques [23], [24] have not yet been fully integrated into an automated telemetry-driven testing pipeline.

These limitations have significant technical, social, economic, ethical, and environmental impacts by reducing software reliability, increasing maintenance costs, weakening user trust, affecting accessibility, and wasting computational resources. This research belongs primarily to the Software Systems & Technologies (SST) research cluster, particularly Software Engineering, Mobile Computing, and Requirements Analysis & Architecture Design, while integrating Artificial Intelligence (CoEAI) and Computing Infrastructure (CI) concepts. It targets the ICT & Software industry, with applications in finance, healthcare, transportation, and public digital services. The proposed framework also contributes to SDG 9 (Industry, Innovation and Infrastructure), SDG 8 (Decent Work and Economic Growth), and SDG 11 (Sustainable Cities and Communities) by enabling more reliable, resilient, and sustainable mobile software systems.

References (IEEE Style)

Component 1 – Context-Aware Environmental Perturbation Engine

- [1] A. M. Abbasi, M. Al-Tekreeti, K. Naik, A. Nayak, P. Srivastava, and M. Zaman, "Characterization and Detection of Tail Energy Bugs in Smartphones," *IEEE Access*, 2018.
- [2] X. Sun, X. Chen, Y. Liu, J. Grundy, and L. Li, "Taming Android Fragmentation through Lightweight Crowdsourced Testing," *IEEE Transactions on Software Engineering*, 2023.
- [3] W. Oliveira and F. Castor, "EBSERVER: Automating Resource-Usage Data Collection of Android Applications," in *2023 IEEE/ACM 10th International Conference on Mobile Software Engineering and Systems (MOBILESoft)*, 2023.
- [4] R. Jabbarvand, J.-W. Lin, and S. Malek, "Search-Based Energy Testing of Android," in *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*, 2019.

[5] X. Li, Y. Yang, Y. Liu, J. P. Gallagher, and K. Wu, "Detecting and Diagnosing Energy Issues for Mobile Applications," *arXiv preprint arXiv:1901.01062v2*, 2019.

[6] J. Marcano, K. Mikowicz, A. B. Rovirosa, A. Samant, T. Smyth, and S. Puligadda, "Scaling Mobile Chaos Testing with AI-Driven Test Execution," *arXiv preprint arXiv:2602.06223v1*, 2026.

Component 2 – Human Stress-Aware Input Interaction Engine

[7] P. M. Fitts, "The information capacity of the human motor system in controlling the amplitude of movement," *Journal of Experimental Psychology*, vol. 47, no. 6, pp. 381–391, 1954, doi: 10.1037/h0055392.

[8] X. Bi, Y. Li, and S. Zhai, "FFitts law: Modeling finger touch with Fitts' law," in *Proc. SIGCHI Conf. Human Factors in Computing Systems (CHI '13)*, Paris, France, 2013, pp. 1363–1372, doi: 10.1145/2470654.2466180.

[9] Y. Ma, S. Zhai, I. V. Ramakrishnan, and X. Bi, "Modeling touch point distribution with rotational dual Gaussian model," in *Proc. 34th Annu. ACM Symp. User Interface Software and Technology (UIST '21)*, Virtual Event, USA, 2021, pp. 1197–1209, doi: 10.1145/3472749.3474816.

[10] Y. Gao, N. Bianchi-Berthouze, and H. Meng, "What does touch tell us about emotions in touchscreen-based gameplay?," *ACM Transactions on Computer-Human Interaction*, vol. 19, no. 4, Article 31, 2012, doi: 10.1145/2395131.2395138.

[11] C. Epp, M. Lippold, and R. L. Mandryk, "Identifying emotional states using keystroke dynamics," in *Proc. SIGCHI Conf. Human Factors in Computing Systems (CHI '11)*, Vancouver, BC, Canada, 2011, pp. 715–724, doi: 10.1145/1978942.1979046.

[12] Z. Liu, C. Chen, J. Wang, M. Chen, B. Wu, X. Che, D. Wang, and Q. Wang, "Make LLM a testing expert: Bringing human-like interaction to mobile GUI testing via functionality-aware decisions," in *Proc. IEEE/ACM 46th Int. Conf. Software Engineering (ICSE '24)*, Lisbon, Portugal, 2024, doi: 10.1145/3597503.3639180.

[13] T. Su, J. Wang, and Z. Su, "Benchmarking automated GUI testing for Android against real-world bugs," in *Proc. 29th ACM Joint European Software Engineering Conf. and Symp. Foundations of Software Engineering (ESEC/FSE '21)*, Athens, Greece, 2021, pp. 119–130, doi: 10.1145/3468264.3468620.

Component 3 – Localized Multimodal Vision-Semantic Oracle

[14] Z. Liu et al., "Seeing is Believing: Vision-Driven Non-Crash Functional Bug Detection for Mobile Apps," *IEEE Transactions on Software Engineering*, vol. 51, no. 12, pp. 3452–3466, 2025, doi:10.1109/TSE.2025.3614469.

[15] J. Wang, Y. Huang, C. Chen, Z. Liu, S.

Wang, and Q. Wang, "Software Testing With Large Language Models: Survey, Landscape, and Vision," *IEEE Transactions on Software Engineering*, vol. 50, no. 4, pp. 911–936, 2024, doi:10.1109/TSE.2024.3368208.

[16] Z. Liu et al., "A Study of Using Multimodal LLMs for Non-Crash Functional Bug Detection in Android Apps," 2024.

[17] X. Li et al., "Extraction and Empirical Evaluation of GUI-Level Invariants as GUI Oracles in Mobile App Testing," *Information and Software Technology*, vol. 177, 2025, doi:10.1016/j.infsof.2024.107531.

[18] A. Baral et al., "Automating GUI-based Test Oracles for Mobile Apps," in *Proceedings of the IEEE/ACM 21st International Conference on Mining Software Repositories (MSR)*, 2024, doi:10.1145/3643991.3644930.

[19] Y. Su et al., "Independent View Fuzzing for Detecting Non-Crashing Functional Bugs in Android Apps,"

Component 4 – Telemetry-Driven Self-Healing Test Synthesis Engine

[20] K. Moran, M. Linares-Vásquez, C. Bernal-Cárdenas, and D. Poshyvanyk, "Automatically Discovering, Reporting and Reproducing Android Application Crashes," *IEEE International Conference on Software Testing, Verification and Validation (ICST)*, 2016.

[21] R. Chen, T. Su, M. Liu, Y. Wang, and Z. Li, "ReCDroid: Automatically Reproducing Android Application Crashes from Bug Reports," *IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2023.

[22] S. Zhang, Y. Liu, and J. Chen, "AdbGPT: Automated Android Bug Reproduction using Large Language Models," *arXiv preprint arXiv:2401.xxxxx*, 2024.

[23] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019.

[24] D. Rost, A. Møller, and M. Schäfer, "Healenium: Self-Healing Selenium Tests using Machine Learning," *Proceedings of the International Conference on Software Engineering Companion (ICSE-C)*, 2021.

6. Brief description of Existing Research and Systems (300 words max)

Summary of Existing Products and Systems

Dominant mobile testing tools include Appium, Android Monkey, and Stoa. Appium executes scripted UI interactions via programmatically perfect coordinate-based clicks. Android Monkey generates random event streams without behavioral intelligence or environmental awareness. Stoa employs stochastic finite automaton-based exploration [Stoa] but still fires geometrically exact inputs under ideal lab conditions. For non-crash failure detection, recent systems such as VisionDroid (2024) [VisionDroid] and Trident introduce AI-assisted visual oracles, but both depend on cloud-hosted multimodal models (e.g., GPT-4o), incurring latency, cost, and privacy risks. For crash reproduction, tools like ReCDroid+ [21] and AdbGPT [AdbGPT] automate bug replay but require structured, human-authored reports.

Summary of Existing Research

Previous studies have investigated individual aspects of mobile robustness testing. Environmental studies demonstrate that runtime conditions such as network behavior, energy consumption, and device fragmentation significantly influence application reliability. Human-computer interaction research models touch behavior using Fitts' Law, Gaussian touch distributions, and affective computing to explain how cognitive stress affects user interaction. Recent advances in vision-language models have improved non-crash GUI fault detection, while self-healing testing techniques and semantic embeddings have increased test maintenance capabilities. However, these research directions remain largely independent and have not been unified into a comprehensive mobile robustness testing framework.

Gaps and Limitations Identified

Five major research gaps remain.

- **Biomechanical Touch Modeling:** No existing mobile testing framework operationalizes validated biomechanical touch models (e.g., Fitts' Law and the Rotational Dual Gaussian model) as a robustness testing mechanism. Existing human-like testing systems such as GPTDroid improve decision-making while still executing geometrically precise inputs.
- **Adaptive Stress-Aware Testing:** Existing exploration strategies, including Stoa, adapt to GUI states during exploration but do not dynamically modify interaction behavior based on runtime performance signals such as latency, frame drops, or Application Not Responding (ANR) events.
- **On-Device Non-Crash Detection:** Recent systems such as VisionDroid successfully detect non-crash GUI failures using multimodal large language models, but they rely on cloud-hosted models, resulting in increased cost, latency, and privacy concerns. To the best of our knowledge, no framework

performs this detection

entirely on-device using a compact quantized Vision-Language Model.

- **Telemetry-to-Test Automation:** Existing crash reproduction systems including ReCDroid, AdbGPT, and CrashTranslator automate bug reproduction from structured bug reports or stack traces. To the best of our knowledge, no existing framework compiles raw production telemetry breadcrumbs (e.g., Sentry or Firebase interaction traces) directly into executable, self-healing regression test scripts.
- **Environmental Perturbation Orchestration:** Although emulator controls, device-state commands, and chaos-engineering techniques support individual environmental manipulations, to the best of our knowledge no existing framework systematically orchestrates multiple environmental perturbations (e.g., network handovers, CPU throttling, battery constraints, and interruptions) within the test execution loop and correlates them with observed application failures.

7. Brief description of the solution's nature, including a conceptual diagram (maximum 500 words).

The proposed solution is a Human Behavior-Aware Robustness Testing Framework for Mobile Applications, designed to improve mobile application reliability by simulating biomechanically and cognitively realistic user behavior, orchestrating multi-factor environmental stress, detecting non-crash failures entirely on-device, and generating self-healing regression tests directly from production telemetry. The framework is built to close the five specific gaps identified in Section 6, rather than to generally improve on existing testing tools.

How the System Addresses Identified Gaps

- Closes the Biomechanical Touch Modeling gap - The Human Stress-Aware Interaction Engine operationalizes validated motor models (FFitts Law's dual-distribution hypothesis [8]; the Rotational Dual Gaussian touch-endpoint model [9]) as an active fault-exposure mechanism, rather than as passive human-likeness at the decision level only, as in GPTDroid [12].
- Closes the Adaptive Stress-Aware Testing gap - Unlike Stoa, which adapts exploration to GUI state but not to runtime performance, the Interaction Engine couples stress-state parameters (frustration, urgency) to real-time performance signals (latency, frame drops, ANR events), so distortion intensity escalates dynamically during testing rather than following a fixed profile.
- Closes the On-Device Non-Crash Detection gap - The Localized Vision-Semantic Oracle performs non-crash failure detection using a compact, quantized Vision-Language

Model running entirely on-device, addressing the cloud-hosted latency, cost, and privacy limitations of VisionDroid and Trident [14].

- Closes the Telemetry-to-Test Automation gap - The Telemetry-Driven Self-Healing Engine compiles raw production breadcrumbs (Sentry/Firebase interaction traces) directly into executable, self-healing Appium scripts, removing the dependency on structured human-authored bug reports required by ReCDroid+ [21] and AdbGPT [22].
- Closes the Environmental Perturbation Orchestration gap - The Environmental Perturbation Engine dynamically coordinates network, CPU, battery, and interruption stressors together, correlated with the app's current execution state (login, transaction, sync), rather than injecting single, static conditions in isolation.
- Together, these four components form a closed-loop pipeline: environmental and behavioral stress is injected, failures are detected on-device, and detected failures are automatically compiled into regression tests continuously improving coverage using real-world signals rather than static test suites.

System Perspective

Input

- Mobile application under test (Android APK / runtime app)
- Real-world environmental conditions (network, battery, CPU, interruptions)
- User interaction parameters (stress state, biomechanical profile)
- Production telemetry logs (Sentry / Firebase breadcrumbs)
- UI screenshots and accessibility hierarchy

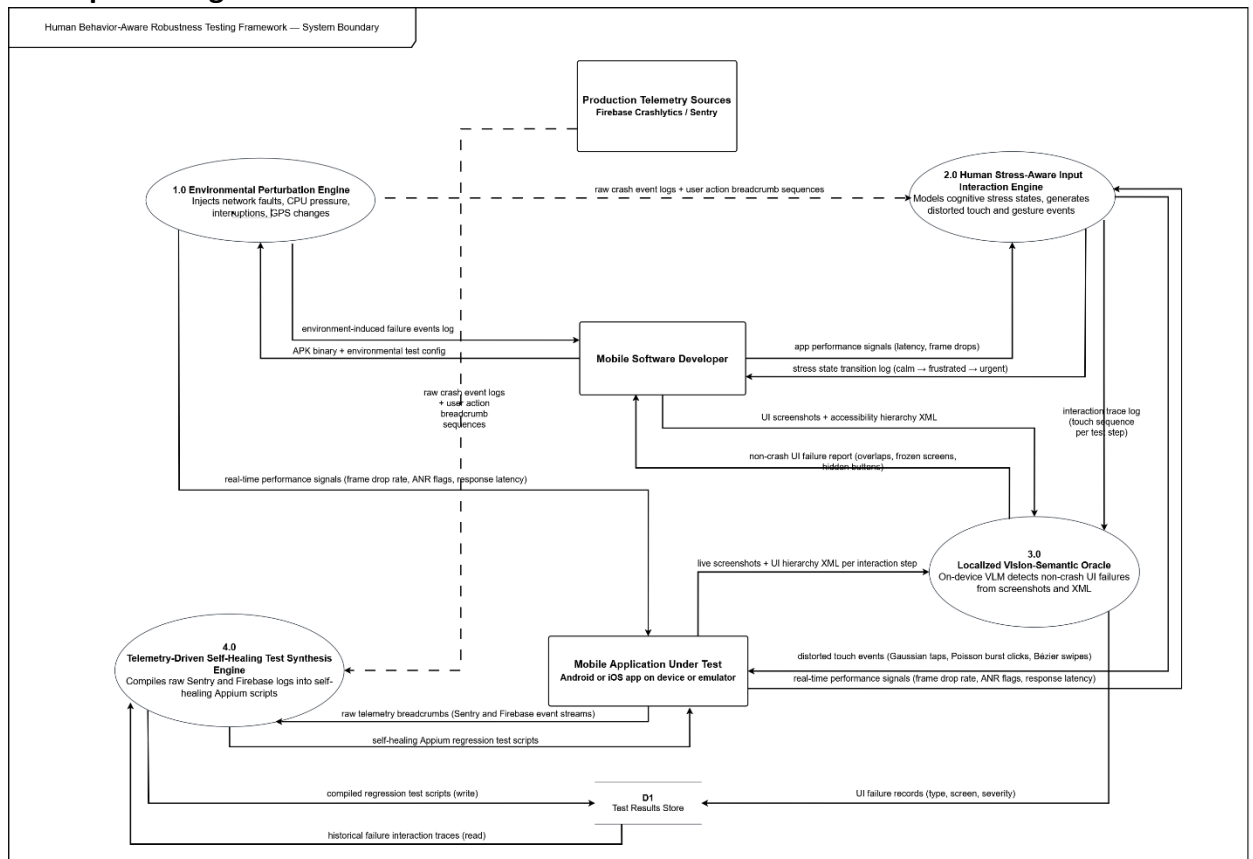
Process - four integrated components:

1. Context-Aware Environmental Perturbation Engine - orchestrates network, CPU, battery, and interruption stress in coordination with app execution state.
2. Human Stress-Aware Interaction Engine - generates biomechanically distorted touch input (FFitts/Rotational Dual Gaussian-based) [8], [9], intensity-coupled to modeled cognitive stress.
3. Localized Vision-Semantic Oracle - on-device quantized VLM jointly analyzing screenshots and view hierarchy to detect overlaps, freezes, and missing elements in real time [14].
4. Telemetry-Driven Self-Healing Engine - converts raw telemetry into executable regression scripts, using Sentence-BERT-based semantic matching [23] for locator self-healing.

Output

- Stress-aware test execution results
- Crash and non-crash detection reports
- Automatically generated, self-healing regression scripts
- Robustness evaluation reports quantifying gap-specific improvements over Appium/Monkey/Stoat [13] baselines

Conceptual Diagram



- Brief overview of the availability of necessary specialized domain expertise, knowledge, and data. (500 words max)

Domain Expertise

The proposed framework draws on four intersecting research disciplines, each with established academic foundations and accessible expert knowledge.

In **mobile software testing**, mature tooling and benchmark infrastructure already exist. The Themis benchmark [13] provides a peer-reviewed, ground-truth set of reproducible crash bugs, developed through two person-months of validation with evaluation spanning over 10,920 CPU hours, an unusually rigorous dataset by mobile-testing standards. Its infrastructure integrates

directly with baseline tools such as Appium and Android Monkey, enabling reproducible comparison for Components 1, 2, and 4.

In **human-computer interaction and biomechanical motor modeling**, well-established theoretical foundations are available. Fitts' Law [8] extends Fitts' Law to account for finger tremor and absolute touch-point precision, directly applicable to Component 2's spatial distortion layer. The Rotational Dual Gaussian Model [9] reflects how touch points elongate along the finger's movement direction, providing a validated mathematical basis for the stochastic noise injection engine.

In **Vision-Language Models for on-device inference**, open-source weights and quantization toolchains are freely accessible. Qwen2-VL supports quantization via AutoGPTQ and AutoAWQ, fine-tuning via Llama-Factory, and deployment via vLLM, with both 2B and 7B variants released under Apache 2.0, lowering the barrier to fine-tuning and compressing the oracle for Component 3.

In **self-healing test automation**, foundational techniques exist but mobile-specific gaps remain. Appium has no built-in locator self-healing, and approaches such as Healenium [24] target web DOM structures rather than mobile challenges like accessibility ID inconsistency, validating the gap addressed by the Sentence-BERT-based [23] self-healing layer in Component 4.

Available Data Sources

Evaluation data is organized as a **portfolio mapped to each component**, comprising in aggregate: 50+ reproducible real-world crash bugs, 72,000+ UI screens from 9,700+ real applications, an open repository of thousands of instrumentable apps, and self-collected production telemetry.

Crash/robustness ground truth (Components 1, 2, 4): Themis [13] provides reproducible crash bugs with baseline-tool integration already built in; roughly a third of its bugs are undetectable by any existing tool, evidencing the detection gap this framework targets.

Non-crash / vision oracle data (Component 3): Rico [25] provides 72,000+ UI screens (66,000+ unique) with paired view hierarchies from 9,700+ apps, used to fine-tune the on-device VLM. Ground truth for non-crash failures is built via synthetic defect injection (overlap, occlusion, missing widgets, frozen states), the same approach used for VisionDroid's injection dataset [14], supplemented by manually verified real non-crash bugs mined from open-source issue trackers.

Scale-out app corpus (all components): the F-Droid repository (f-droid.org) provides an extensible pool of instrumentable apps beyond the benchmark set.

Production telemetry (Component 4): Firebase Crashlytics and Sentry event streams, collected from instrumented demo apps operated by the team, provide raw telemetry for the telemetry-to-test compiler without dependency on proprietary data.

Per-component evaluation metrics, benchmarked against the literature cited above, are detailed in Section 9.

Summary

All four components are built on accessible, open-source tools, publicly available benchmarks, and well-documented research foundations. No proprietary data or restricted hardware is required, making the full framework feasible within a standard university research environment.

9. Objectives and Novelty

Main Objective

To design and develop a comprehensive Human Behavior-Aware Robustness Testing Framework for Mobile Applications that simulates realistic user interactions under cognitive stress and real-world environmental conditions, detects both crash and non-crash failures using on-device intelligent analysis, and automatically generates self-healing test cases from production telemetry data in order to improve mobile application reliability and robustness.

Member Name with Registration No	Sub Objective	Tasks	Novelty
IT23151260 G.L.S.CHANLAKA	Develop a Context-Aware Environmental Perturbation Engine	- Simulate network variability (WiFi ↔ mobile data handovers, latency, packet loss, bandwidth throttling), injected at configurable semantic points in test flows (e.g., mid-transaction), extending static context fuzzing approaches - Inject device-level stress (CPU load, memory pressure, low-battery states, thermal throttling profiles) synchronized with the test-execution loop - Simulate real-world interruptions (calls, SMS, notifications, backgrounding,	While prior work has tested apps under varied static contexts notably Caiipa's large-scale contextual fuzzing and individual condition controls exist in emulators and chaos-engineering practice, to the best of our knowledge no testing framework ORCHESTRATES environmental perturbations that are (i) injected at semantically meaningful points within the live test-execution loop (e.g., mid-transaction network handover, thermal throttling under load),

		Doze/App-Standby transitions) and context changes (GPS shifts, rotation, brightness, dark mode) • Implement a perturbation ORCHESTRATOR/scheduler that (a) triggers conditions based on test-step semantics and real-time performance signals, and (b) logs every perturbation event with timestamps for environment-to-failure correlation • Produce an environment-induced failure attribution report by correlating the perturbation event log with detected crash and non-crash failures on the Themis benchmark apps	(ii) coupled to a behavior-aware testing agent and real-time performance signals, and (iii) systematically correlated with the resulting crash and non-crash failures to produce environment-to-failure attribution.
IT23156760 L.T.JAYAWARDHANA	Build a Human Stress-Aware Input Interaction Engine that generates biomechanically and cognitively realistic touch input as a systematic fault-	• Model cognitive stress states (frustration, urgency, distraction), with state parameters (tap frequency, touch duration/pressure proxies, spatial deviation, interaction timing) derived from published empirical distributions in the affective-	To the best of our knowledge, no existing automated testing agent operationalizes validated biomechanical touch models (FFitts Law, dual-Gaussian endpoint distributions) as a fault-exposure mechanism, nor conditions input distortion parameters on empirically grounded cognitive stress states Existing human-like

	exposure mechanism	touch literature, rather than invented values • Implement spatial touch distortion using dual-Gaussian touch endpoint distribution models (FFitts dual-distribution hypothesis; Rotational Dual Gaussian model) • Apply temporal randomness using Fitts'/FFitts movement-time models combined with stochastic event timing • Generate gesture-level distortions (shaky swipes, incomplete drags, Bezier-path deviations) consistent with the direction-elongated touch distributions in • Couple stress state with interaction intensity, so distortion parameters shift dynamically as the modeled stress level changes (e.g., frustration escalation after repeated failed taps "rage clicking")	testing approaches (e.g., GPTDroid) achieve human-likeness at the decision level while still executing geometrically perfect inputs; this work introduces stress-conditioned distortion at the motor-execution level as a systematic robustness testing mechanism.
--	-----------------------	--	--

IT23452916 M.L.BOOSO	Develop a Localized Multimodal Vision-Semantic Oracle	• Fine-tune lightweight Vision-Language Model (VLM) for mobile UI analysis • Apply quantization (4-bit/8-bit) for on-device execution • Analyze screenshot + UI hierarchy jointly • Detect non-crash UI failures (overlap, freeze, hidden elements) in real time	First fully on-device non-crash failure detection system using a compact VLM, eliminating dependency on cloud-based expensive models while preserving real-time UI validation capability.
IT23192300 H.N.MADUBASHINI	Build Telemetry-Driven Self-Healing Test Synthesis Engine	• Extract user interaction flows from production telemetry (e.g., Sentry, Firebase Crashlytics) • Generate automated Appium test scripts using sequence models • Detect UI structural changes across application versions using AST analysis. • Apply Sentence-BERT semantic similarity to automatically remap invalid UI locators and repair broken test scripts.	Introduces an end-to-end telemetry-driven test synthesis pipeline that automatically transforms raw production telemetry into executable Appium regression tests and continuously maintains them using AST-based structural analysis and semantic embedding-based self-healing, eliminating manual bug reproduction and reducing regression maintenance effort.

**10. This section is to be completed by the
Supervisor and the Co-supervisor of the Project.**

- a) This research topic possesses a comprehensive scope suitable for a final-year project.

Yes		No	
-----	--	----	--

- b) The proposed topic exhibits novelty.

Yes		No	
-----	--	----	--

- c) The student group can successfully execute the proposed project.

Yes		No	
-----	--	----	--

- d) The proposed sub-objectives reflect the students' areas of specialization.

Yes		No	
-----	--	----	--

- e) Data required for the project is available.

Yes		No	
-----	--	----	--

- f) Any other comments:

--

	Title	First Name	Last Name	Signature
Supervisor*				
Co-Supervisor*				
External Supervisor				
Summary of external supervisor's (if any) experience and expertise				

*- At this stage, identifying the Co-Supervisor is compulsory. Students should reach out to a staff member, with guidance from their supervisor, to serve as the co-supervisor for this project.